isp-insoft

Augmented Development.

Mensch steuert Agenten

Oliver Krylow

00 · Frage

Werden wir ersetzt?



There is no "I" in LLM

LLM + Harness = Agent

LLM Modell + Software, die Tools bereitstellt und ausführt.

„Agentic Development“?

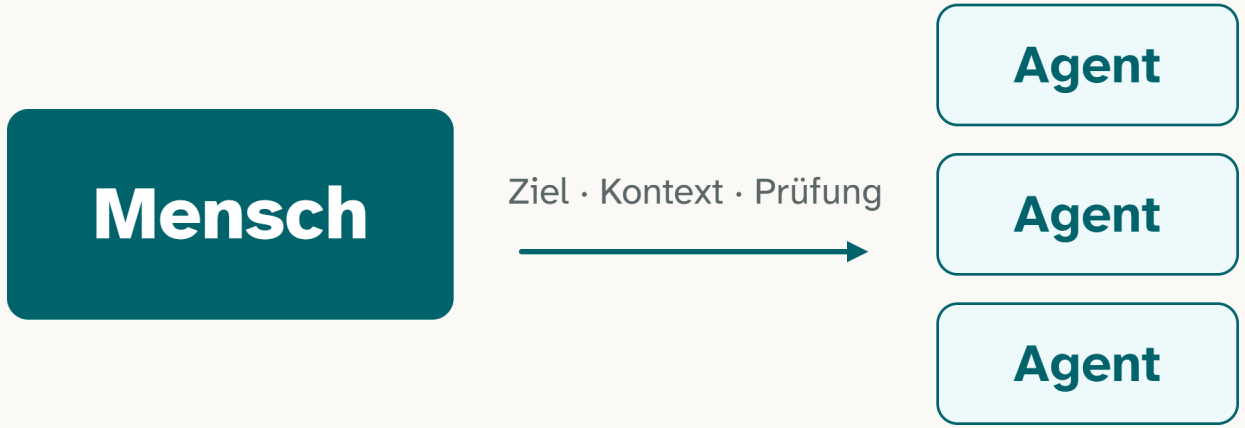
Künstliche **I**ntelligenz **f**ür **f**achliche **E**ntwicklung **n**utzen

---

Artificial **I**ntelligence **D**evelopment **S**ystem

Augmented Development

Mensch steuert Agenten.

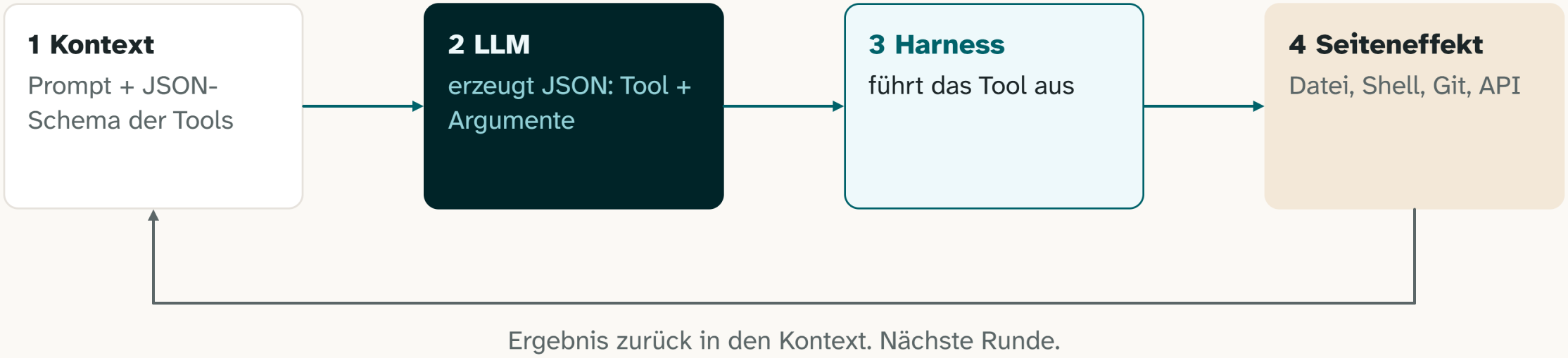


LLM: Text rein. Text raus.



Ein Token nach dem anderen. Anhängen. Wiederholen.

Harness: Werkzeuge per JSON.



LLM + Harness = Agent.

Was bleibt.

1 **Systematisches Denken**

2 **Logische Problemlösung**

3 **Interesse fürs Detail**

Mit Agenten: wichtiger denn je.

ASM → C ≈ Code → Agent?

Hinkt.

Der C-Compiler ist deterministisch und vorhersehbar.

Ein LLM nicht.

Passt.

Auch gute C-Programmierer brauchen ASM-Grundlagen.

Das Prinzip bleibt.

Low-Level-Verständnis

war, ist und bleibt einer der wichtigsten Skills für Seniors.

“The purpose of abstraction is not to be vague, but to create a new semantic level in which one can be absolutely precise.”

Edsger W. Dijkstra · The Humble Programmer (EWD340) · 1972

Natürliche Sprache ist keine solche Ebene.

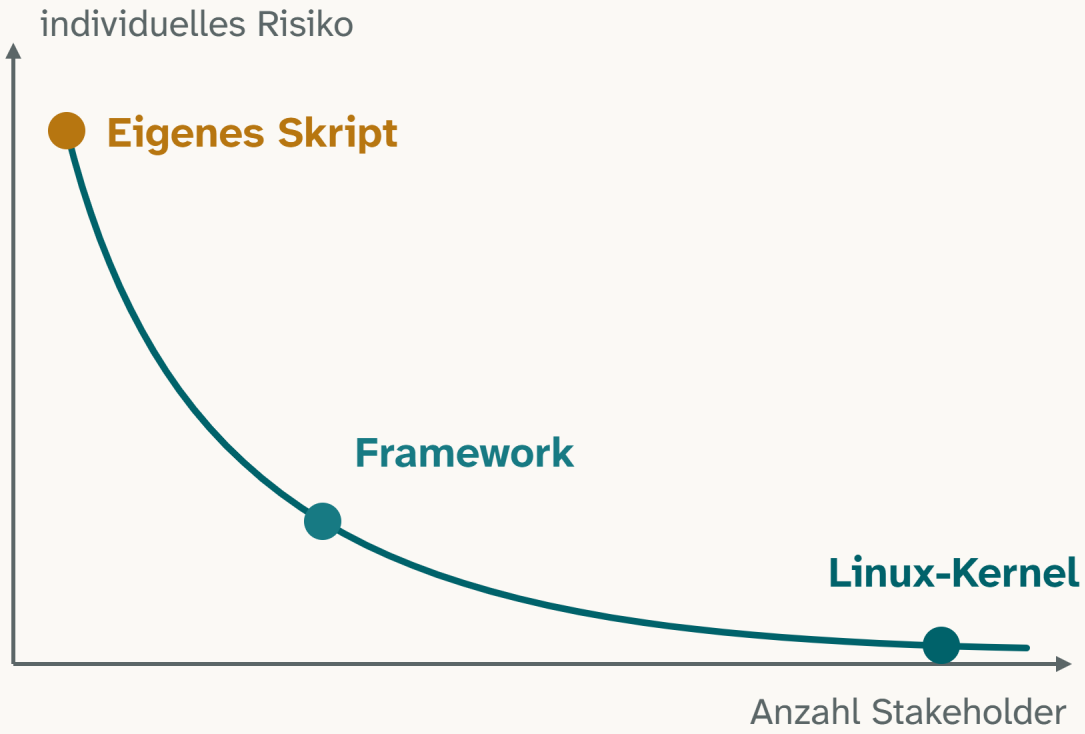
Software ist ein soziales Netz.

App
Frameworks
Bibliotheken
Container
Betriebssystem
Hardware

Warum vertraut die App allem darunter?

Community.
Erfahrung.
Sozialer Graph.

Vertrauen ist Risikoabwägung.



Mehr Stakeholder = geteiltes Risiko.

Geht etwas schief, tragen viele den Schaden. Der Einzelne wenig.

04 · Vertrauen

Agenten sind neu.

Vertrauen baut sich erst auf. Risiken sind unklar. Also:

1

Automatisieren für Qualität

nicht für Menge.

2

LLMs als Lernwerkzeug

Experte + Disziplin. Der intellektuellen Faulheit widerstehen.

3

Verifizierbare Qualitätsmetriken

auf höherer Ebene: Tests, Typen, Mutation, Nutzen beim Kunden.

AI = Automation.

Der wahre Wert: Prozesse automatisieren.

LLMs sind nicht intelligent.

1

Ohne Impuls von außen: keine Frage, keine Handlung.

2

Weiß nicht, welche Prozesse wir automatisieren wollen.

3

Fühlt keinen Schmerz, wenn ein System schlecht läuft.

Coding ist ein Prozess. Von vielen.

Anforderungen	Planung	Architektur	**Coding**
Review	Test	Release	Betrieb
Support	Dokumentation	Schulung	Wartung

Augmented Engineer

Optimiert Agenten für Geschäftsprozesse.

Baut große Systeme, in denen Menschen und Agenten zusammenarbeiten.



Woraus eine Automation besteht.

Harness

Pi, Claude Code, Codex CLI

+

Modell

Claude, GPT, lokale Modelle

+

Tools / MCPs / Skills

Shell, Git, gh, Browser, MCP-Server, SKILL.md

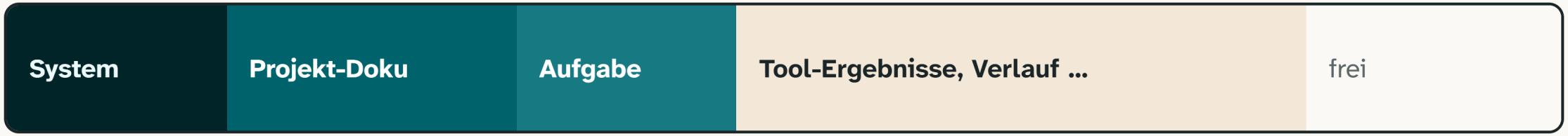
+

Compute

Laptop, CI-Runner, Container

Dazu zwei Grundlagen: **Kontextmanagement** und **selbstverifizierende Loops**.

Kontext ist begrenzt.



Progressive Disclosure

CLAUDE.md, AGENTS.md, Skills: nur laden, was gebraucht wird.

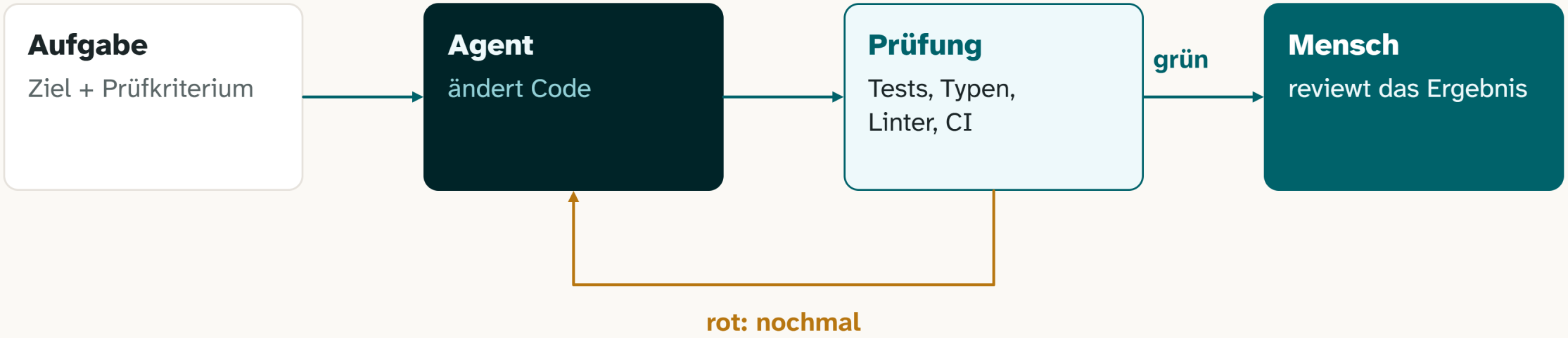
Compaction

Bei ca. 80 % zusammenfassen.

Subagenten

Teilaufgabe mit frischem Kontext.

Selbstverifizierende Loops.



Die Prüfung muss maschinell sein. Sonst gibt es keinen Loop.

CI-Agenten: kaufen oder bauen?

- Code-Review
- Security-Scan (TSS-Anforderung)
- Doku aktuell halten
- Desloppyfier (Anti-Patterns)
- Bug-Fixer (Fix pro Ticket)

Kaufen

z. B. CodeRabbit

Bauen

z. B. toktrak-Golems

Mischen geht auch.

Ein Golem = eine Markdown-Datei.

```
---
harness: claude
model: fable
effort: high
schedule: [wed]
timeout: 45m
verify: mise run verify
---
# QA maintenance
Improve the weakest valuable part
of the test suite or quality tooling.
```

Wer: Harness + Modell

Wann: Wochentag

Prüfung: deklariert

Was: ein Absatz Prompt

06 · Beispiel: toktrak

Eine Woche Golems.

Mo	Di	Mi	Do	Fr	Sa	So
Bugs Pi	**Security** Codex CLI	**QA** Claude Code	**Doku** Pi	**Refactor** Pi **Tester** Claude Code	**Perf** Pi	**Wartung** Pi

Drei Harnesses, Abo-Authentifizierung. Ein Golem pro Aufgabe, ein Branch pro Golem.

Ein Golem-Lauf.

1
Zeitplan
täglich 09:17 UTC, GitHub Actions

→

2
Worktree
isoliert, ein Branch pro Golem

→

3
Agent
Harness + Modell, max. 45 min

→

4
Guards
Prüfung grün, Steuerung unverändert, Workspace sauber

→

5
Pull Request
Mensch reviewt und merged

Merge bleibt manuell.

07 · Zurück zur Frage

Werden Entwickler ersetzt?

Ja.

Wer sich nicht anpassen kann, überlebt nur in sehr speziellen Nischen.

Nein.

Seniors werden mehr denn je gebraucht: systematisches Denken, Problemanalyse, Architektur.

Offene Fragen.

> **Wie bilden wir die nächste Generation aus?**
>
> Keine Ahnung. The kids will be alright.

> **Brauchen wir künftig weniger Entwickler?**
>
> Vielleicht. Keine Ahnung. Die Nachfrage nach neuer Software ist nicht abschätzbar.

08 · Weiterführend

Zum Weiterlesen.

Wer	Was
Stuart Halloway	How to make things, and why · Clojure/conj 2026 (Talk)
Brian Cantwell Smith	The Promise of Artificial Intelligence: Reckoning and Judgment · 2019
Edsger W. Dijkstra	The Humble Programmer (EWD340) · On the foolishness of „natural language programming“ (EWD667)
Fred Brooks	No Silver Bullet · 1986
Peter Naur	Programming as Theory Building · 1985
Joel Spolsky	The Law of Leaky Abstractions · 2002
isp-insoft	github.com/isp-insoft-gmbh/toktrak · .golems/

Danke.

Fragen?

isp-insoft